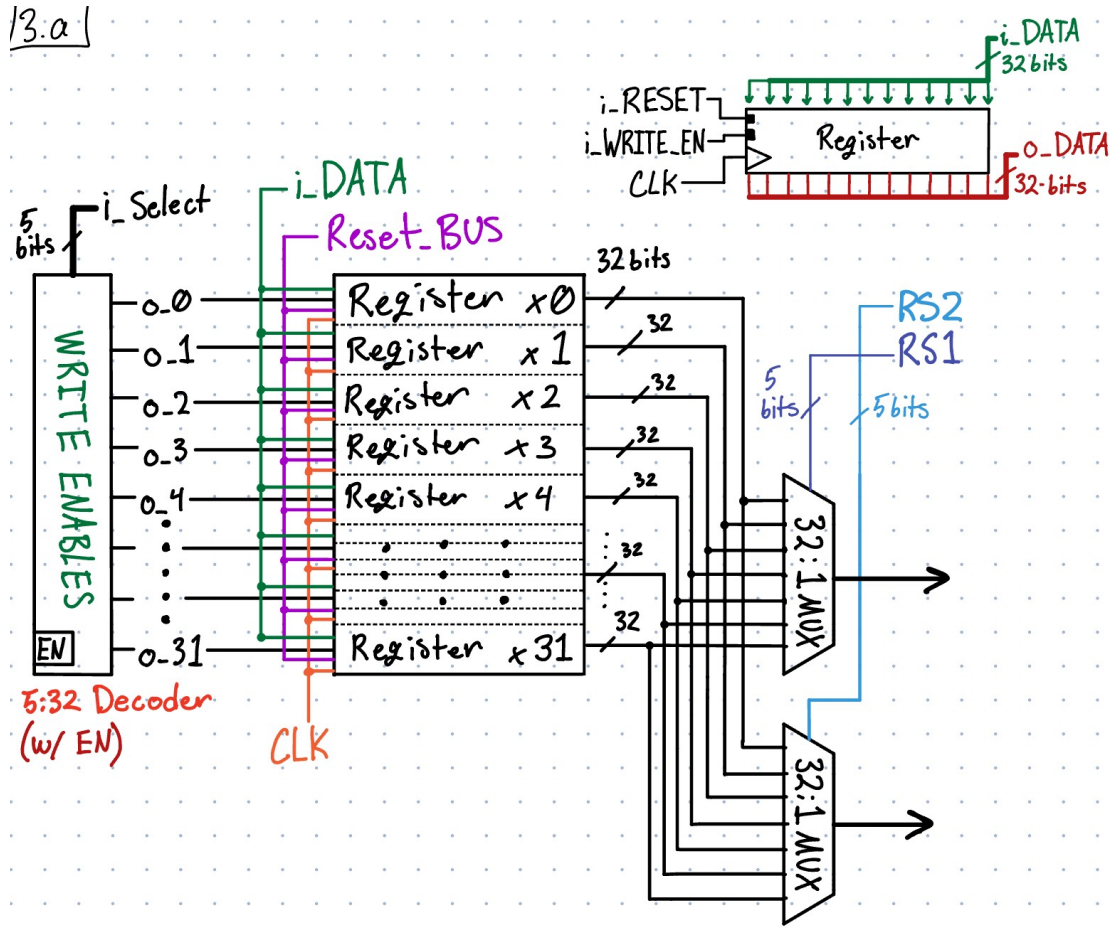
CprE 381, Computer Organization and Assembly-Level Programming

Lab 2 Report

Student Name _____Isaiah Pridie _____

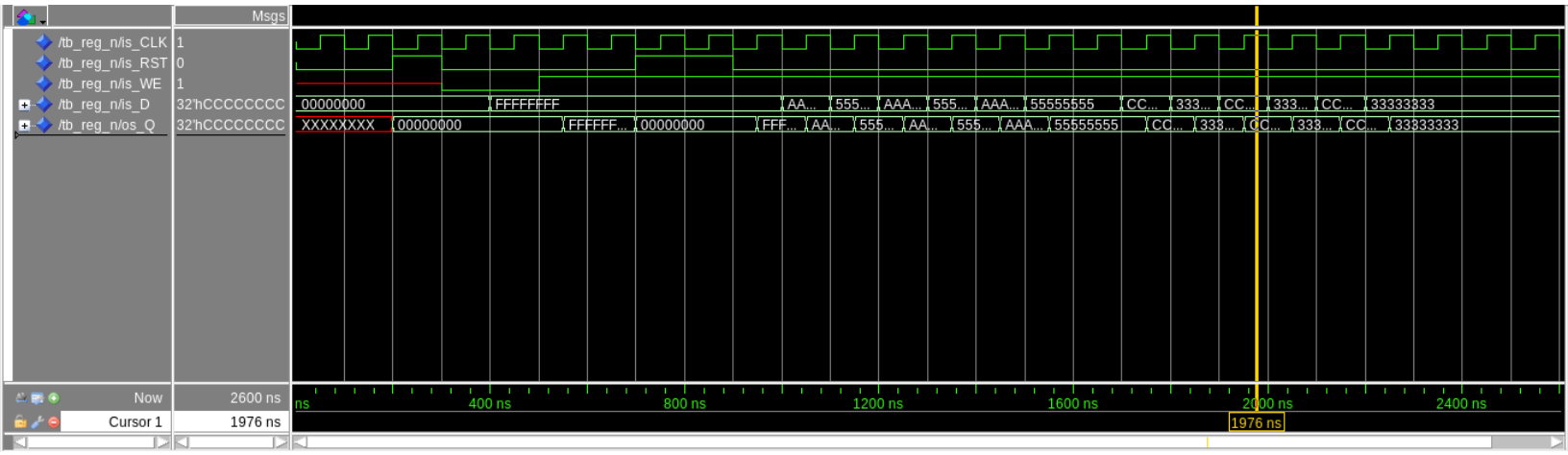
Submit a typeset pdf version of this on Canvas by the due date. Refer to the highlighted language in the lab document for the context of the following questions.

[Part 3 (a)] Draw the interface description (i.e., the "symbol" or high-level blackbox) for the RISC-V register file. Which ports do you think are necessary, and how wide (in bits) do they need to be?
Ports needed for the entire register file is Data_in, Data_out, Clock signal, dest. Reg input (i_WE), 2 register arguments (RS1, RS2), input Reset signal, Write enable (decoder) enable.



[Part 3 (b)] Create an N-bit register using this flip-flop as your basis.
Done.

[Part 3 (c)] Waveform.

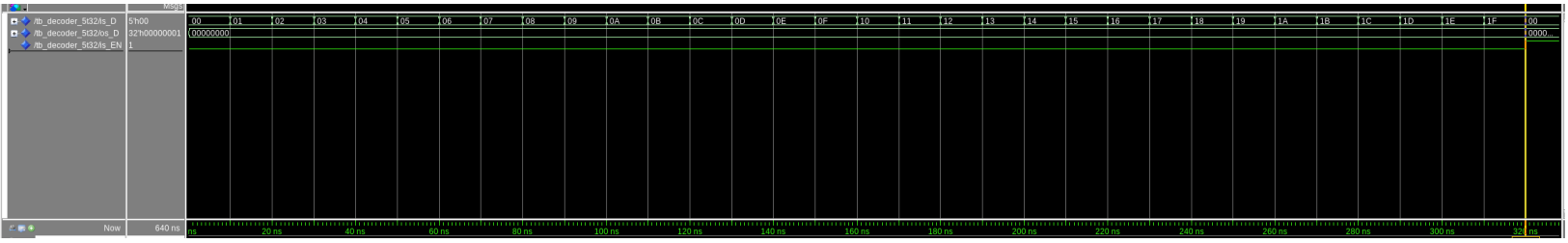


[Part 3 (d)] What type of decoder would be required by the RISC-V register file and why?

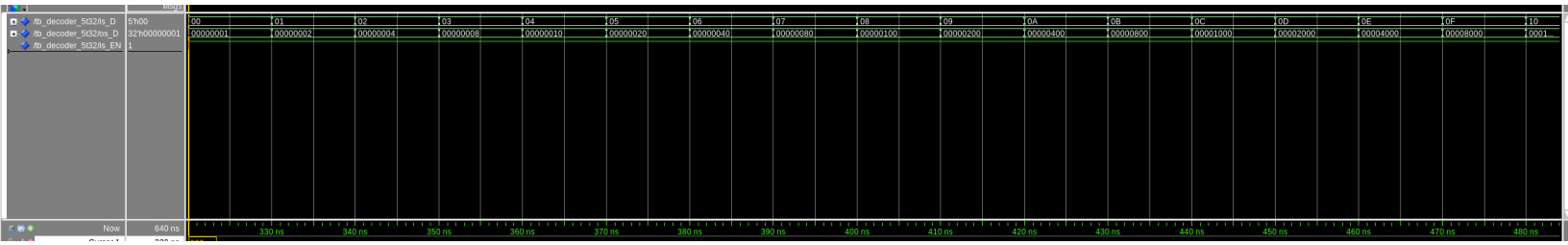
The decoder required is a **5 bit to 32 bit decoder**. This is because when given an instruction, our computer needs to know which register is the destination register.

In machine code the rd (destination register) is represented with 5 bits. Those 5 bits go to a decoder and makes 1 of 32 output signals hot. Each signal goes to a different register. The hot signal enables the write enable on that register, allowing the result of the instruction to be written and saved to that register.

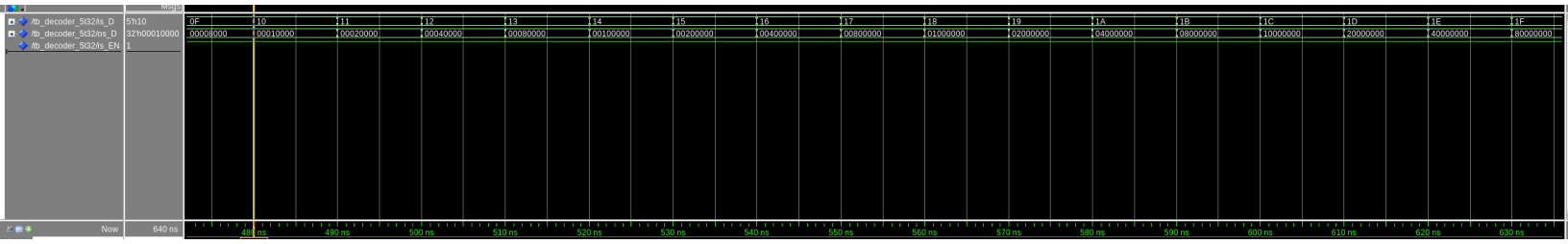
[Part 3 (e)] Waveform.



First screenshot shows all outputs are 0 when enable bit is 0.



Second screenshot shows ~first half of possible inputs and the decoders outputs when EN = 1.



Third screenshot shows ~second half of possible inputs and the decoders outputs when EN = 1.

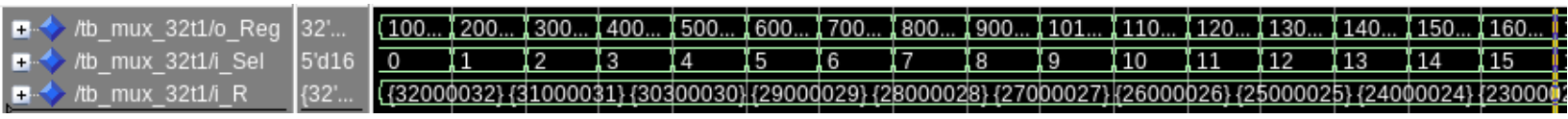
[Part 3 (f)] In your write-up, describe and defend the design you intend on implementing for the next part.

I decided to build my mux similar to how I built my decoder in the fact they are both implemented in dataflow. The easiest way on paper to implement this mux is structurally with a tree of 2:1 muxes. However, in VHDL it is slightly more complicated because you have to use generate functions, and then connect ports to each other on each layer, and what not.

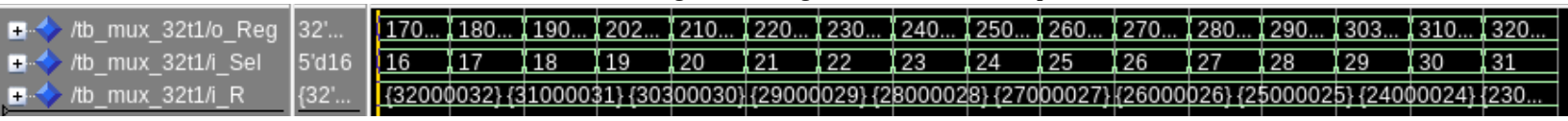
From creating the decoder, I know a mux is basically the same as a decoder. The only main difference is that one usually allows signals/inputs to pass through while another generally generates outputs a signal. It was much easier and probably more efficient to use dataflow and create it this way.

When you build structurally, you are telling VHDL "it needs to be built like this." But when you leave the specific structure of it open ended, the software is allowed to generate what it looks like at hardware level. Most of time the software will generate a faster implementation than what a novice VHDL coder can.

[Part 3 (g)] Waveform.

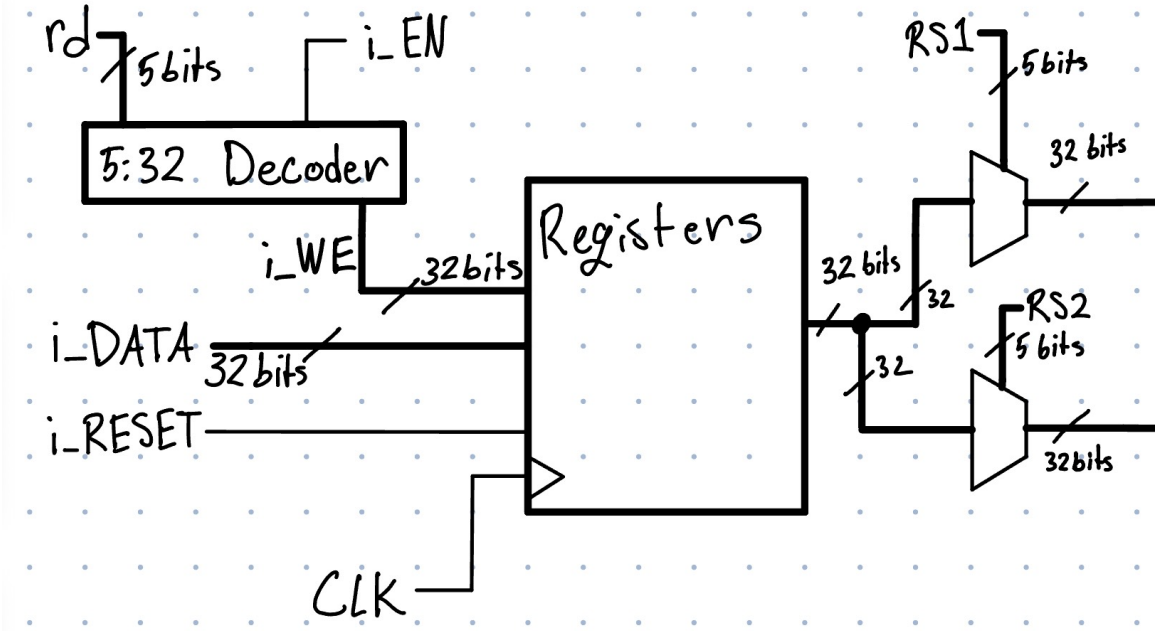


First screenshot shows lower half of the 'registers' being selected and their output.



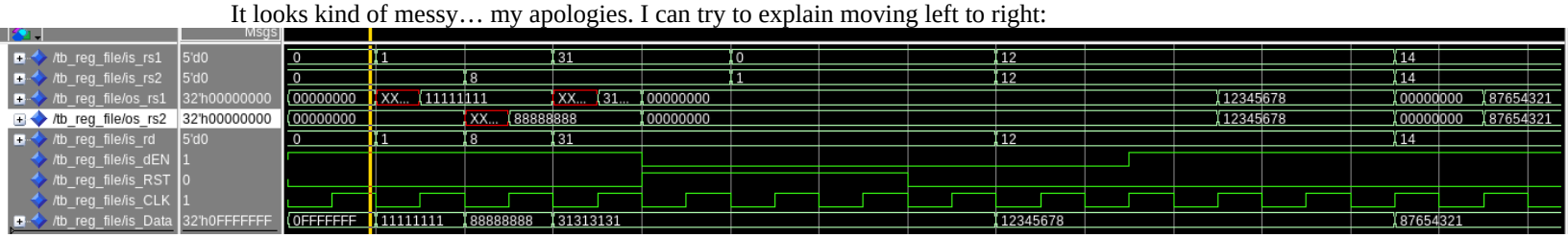
Second screenshot shows upper half of the 'registers being selected and their output.

[Part 3 (h)] Draw a (simplified) schematic (i.e., components within the high-level blackbox) for the RISC-V register file, using the same top-level interface ports as in your solution describe above and using only the register, decoder, and mux VHDL components you have created.



[Part 3 (i)] Waveform.

It looks kind of messy… my apologies. I can try to explain moving left to right:



- Start with NO RESET and Decoder EN ON
- Try to write 0x0FFFFFFF to x0. Assign rs1, rs2, and rd to x0. We see x0 is still 0

- Look at register x1 with rs1. Try to write 0x11111111 to x1 and see if it worked.
- Shows up as unknown first because no reset yet, but turns into 0x11111111

- Look at register x8 with rs2. Try to write 0x88888888 to x8 and see if it worked
    - Similarly to the last one, shows up as unknown first but then shows 0x88888888.

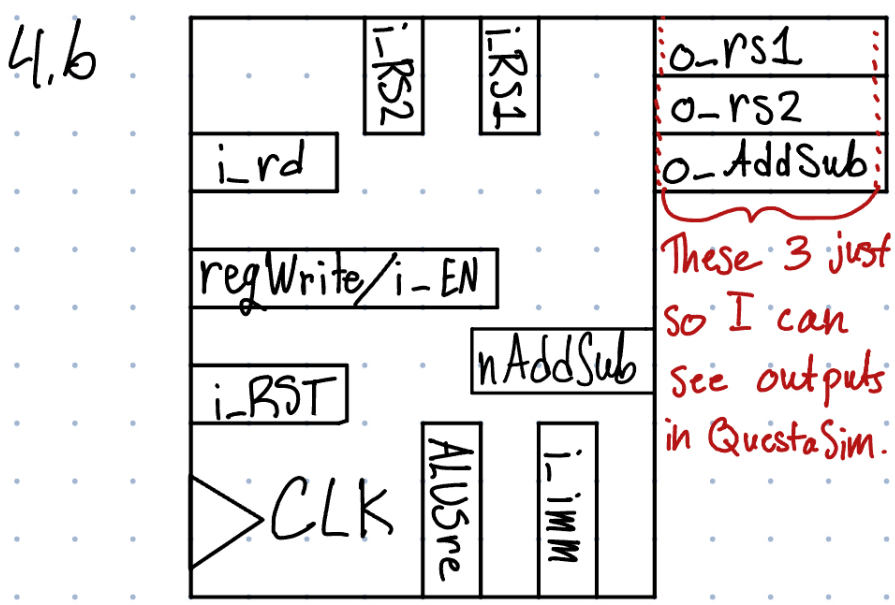
- Look at register x31 with rs1. Try to write 0x31313131 with x31. Same here; unknown → works.

- Turn RESET signal to 1. rs1 shows x31 immediately go to 0. rs2 shows x8 immediately go to 0.

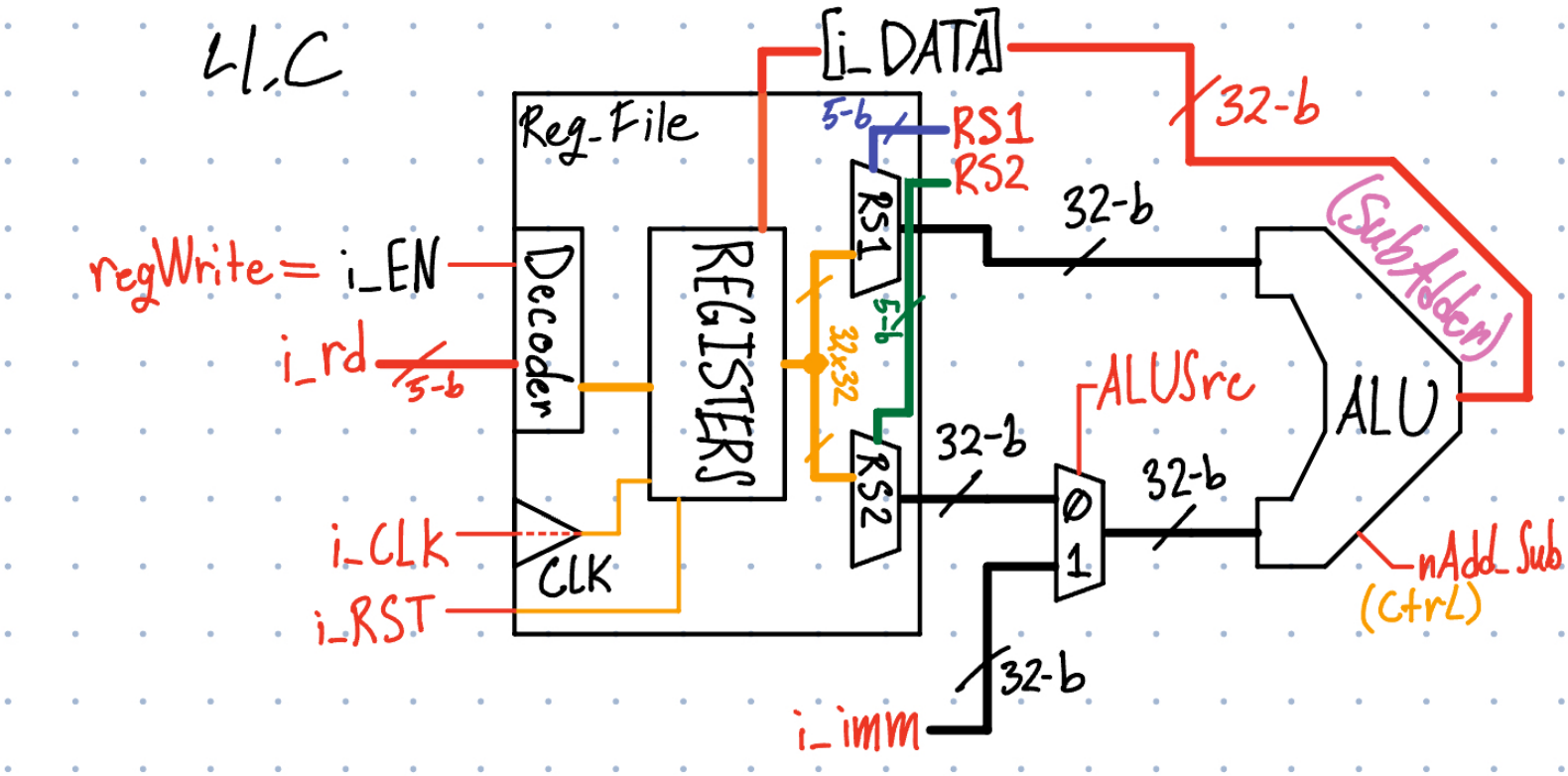
- Set rs1 and rs2 to x12. Try to write 0x12345678 when decoder EN is 0. Then turn on decoder EN and skip a clock cycle. Output did not show up until decoder EN turned back on and hit positive edge a while later.

- Set rs1 and rs2 to x14. Try to write 0x87654321 when decoder EN is 1, but skip a clock cycle (See if output with no clock).
    - Apparently you cannot control when the clock outputs unless you stop the clock all together. So this last part didn't work. It updated the register contents immediately. As a result it immediately output those results.

[Part 4 (b)] Draw a symbol for this RISC-V-like datapath.

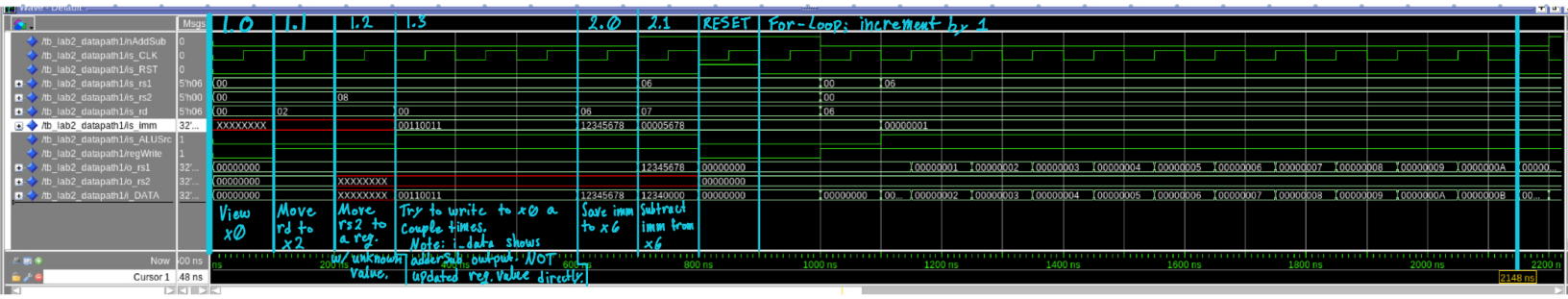


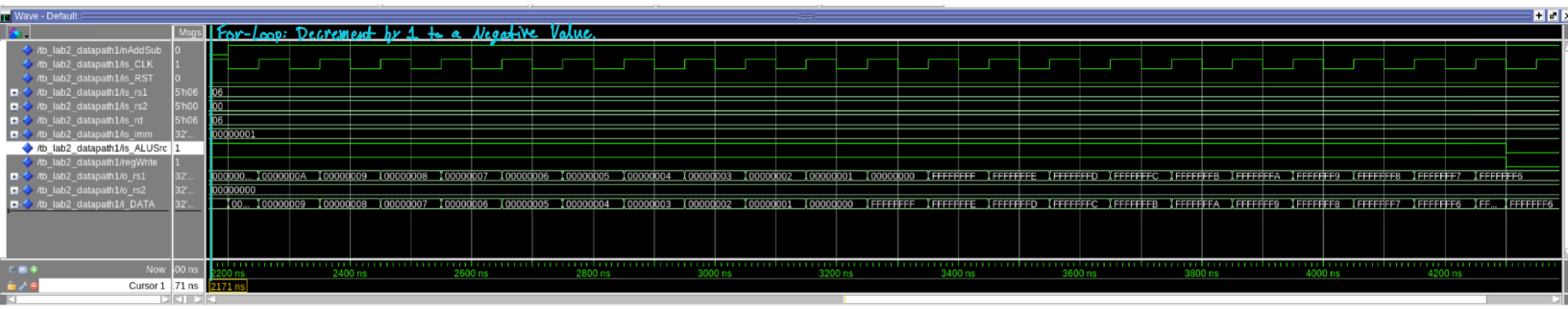
[Part 4 (c)] Draw a schematic of the simplified RISC-V processor datapath consisting only of the component described in part (a) and the register file from problem (1).



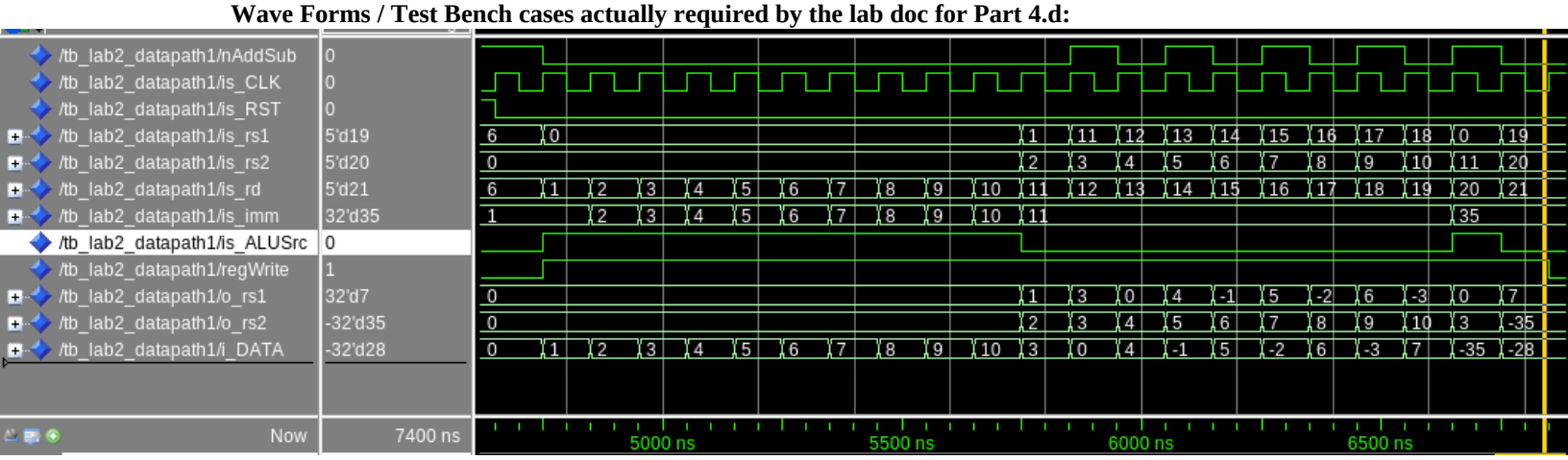
Note: I replaced i_DATA with o_Data. I use that as a signal so I can see on my FINAL waveform for part 7 what value is getting past the mux (part 7) and into the registers (if regWrite is enabled).
Note: I did add a BusMux for part 7 so I can choose between the ALU output and the memory output.

[Part 4 (d)] Include in your report waveform screenshots that demonstrate your properly functioning design. Annotate what the final register file state should be.





Wave Forms / Test Bench cases actually required by the lab doc for Part 4.d:



[Part 5 (a)] Read through the mem.vhd file, and based on your understanding of the VHDL implementation, provide a 2-3 sentence description of each of the individual ports (both generic and regular).

ADDR_WIDTH: Address width for the rows of memory available. This value is 10. So 2^10 = 1024 lines of memory I can read/write to. This is also natural so it can store the value of an unsigned int and cannot be negative.

DATA_WIDTH: A constant (natural, so the size of an (unsigned) int and cannot be negative) that represents the width of the memory (32 bits across) in the 2d array. Each row (32 bits (1 word))

clk: Standard clock input. Clock drives loading data into the ram on the positive clock edge and when the write enable is 1.

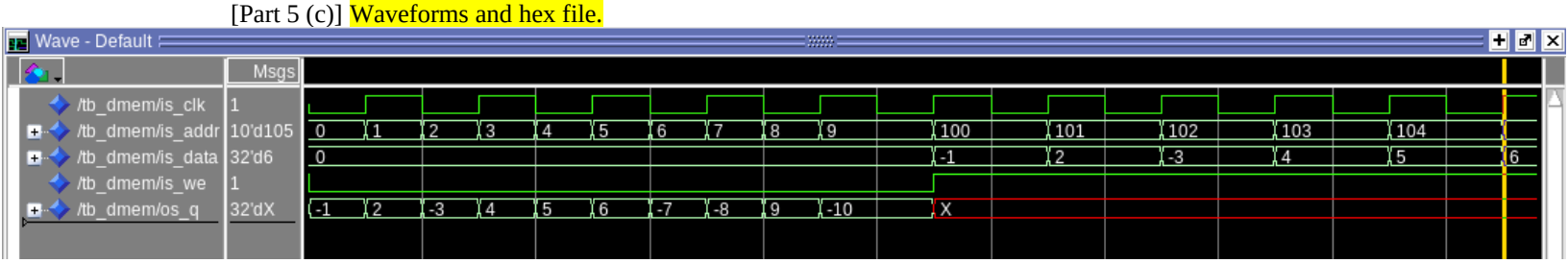
adder: Addr is the address (line) of the row of RAM that will be read from and/or written to. Write will wait for the positive edge of the clock. Reading does not depend on the clock; at the bottom of the file, it shows an if statement with saving data to ram. And outside the clock cycle, unconditionally, q is assigned to the ram at the same address with q being the output.

data: The input being written to the RAM by assigning it to the RAM signal.

we: This is the write enable to the RAM. When we = 1, the row of ram at address ‘adder’ can be overwritten.

q: is the output bus from the RAM signal. This is used to load/copy a value from RAM elsewhere in the CPU. Data being read to RAM signal on the clock cycle.

[Part 5 (c)] Waveforms and hex file.



First half ^^^

Second Half ^^^

Quick Note: q *was* outputting the write value as well before part c.

Looking at mem.vhd, it shows the q assignment from ram is outside of the if statements for clk and addr. So q should be updating regardless of where the clock is at as long as it is after ram has been updated.

But for some reason it no longer updates when writing. BUT if I put a wait for X ns, q will update on the positive edge of the clock for some reason. Then after X ns, it will go back to saying xxxxxxxx because the address changed to the next address. Just wanted to throw this. Idk I am getting that behavior.

[Part 6 (a)] What are the RISC-V instructions that require some value to be sign extended? What are the RISC-V instructions that require some value to be zero extended?

All instructions with an immediate input need to be sign extended to 32 bits.

- All “I” formatted instructions need to be sign extended, even the unsigned instructions; the unsigned part refers to the comparison, not the actual immediate input value.
  - Only exception in the I format instructions are the shift left or right instructions and CSR instructions. The former (shift) instructions get zero extended.
- All “S” format instructions (stores) need to be sign extended.
- All “B” format instructions (branch) need to be sign extended.
- All “J” format instruction (jump) need to be sign extended.

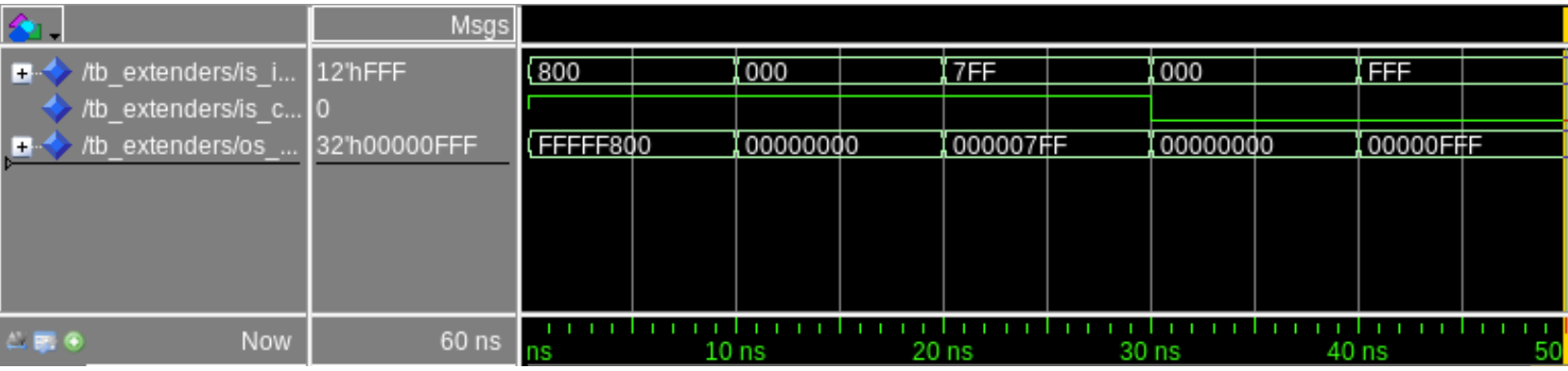
The only instructions that are 0 extended are shift instructions and the CSR instructions.

[Part 6 (b)] what are the different 12-bit to 32-bit “extender” components that would be required by a RISC-V processor implementation? (Zach says change 16 to 12?)

Maybe a mux to choose between either zero extending and sign extending. But this can be done with logic (i.e. if statements).

And not much else. For sign extending it is just concatenating a 20 bit bus filled with the same bit value as the MSB bit of the 12 bit immediate. For zero extending it is just concatenating the 12 bit immediate with 20 bits of zero.

[Part 6 (d)] Waveform.



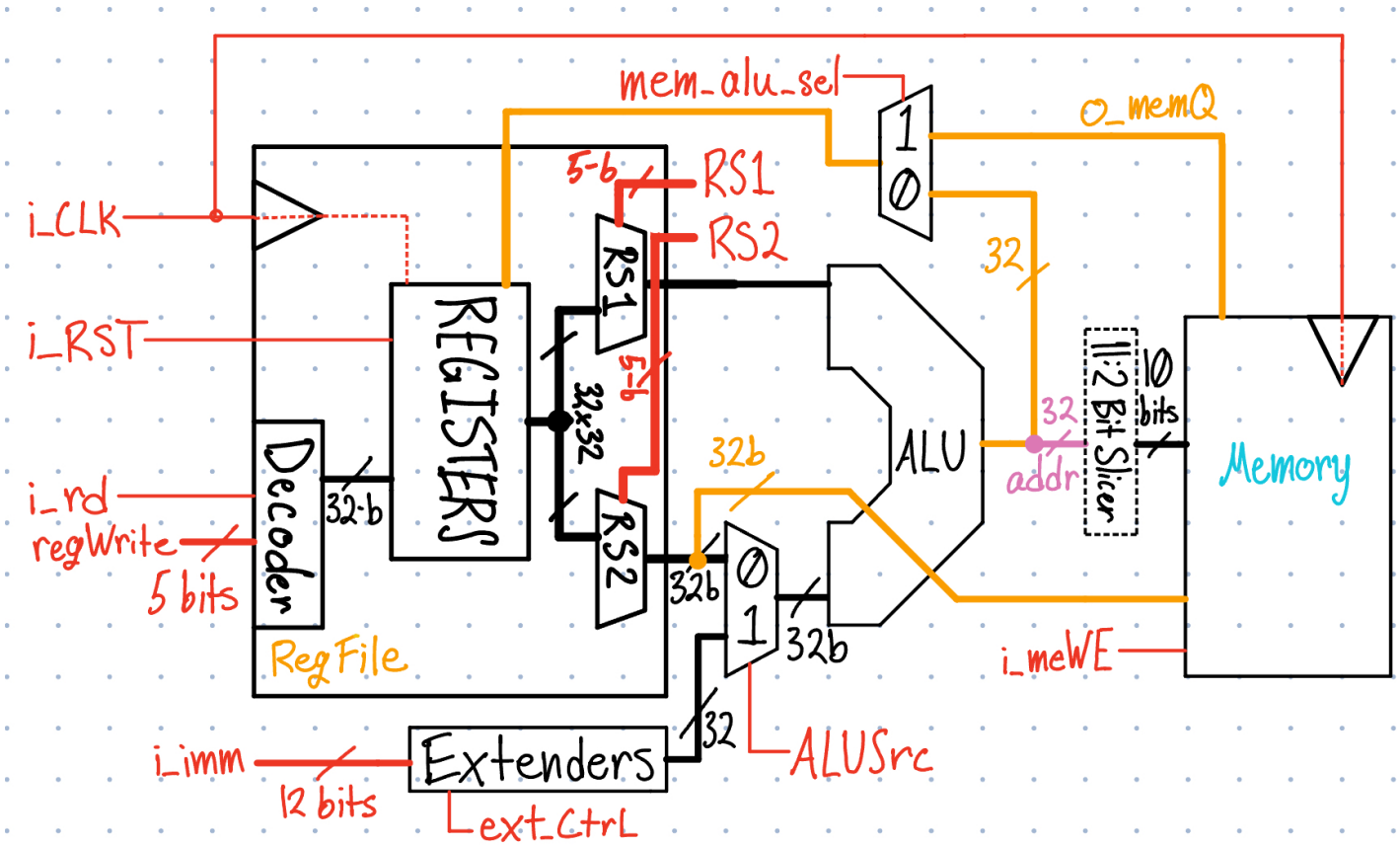
[Part 7 (a)] what control signals will need to be added to the simple processor from part 2? How do these control signals correspond to the ports on the mem.vhd component analyzed in part 3?

Some control signals that need to be added are the:

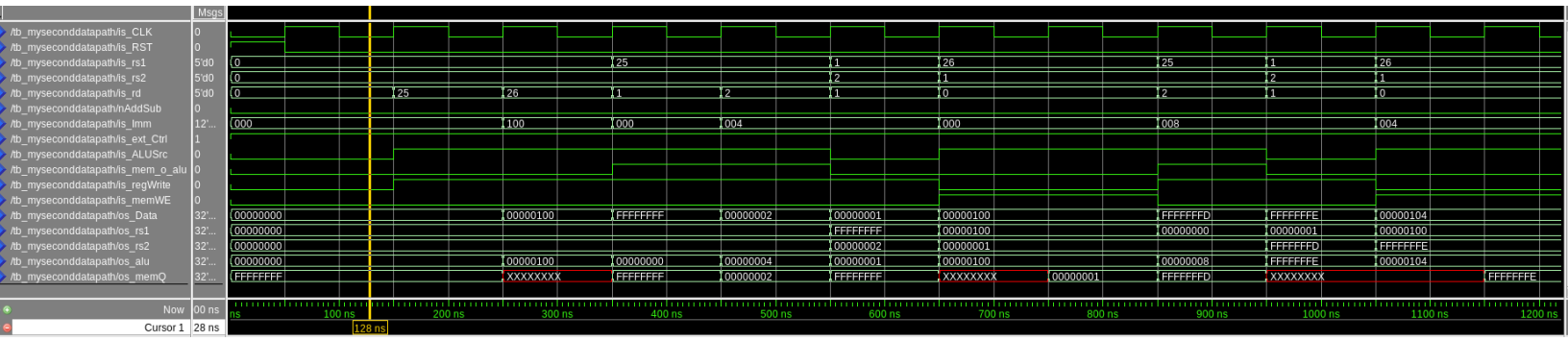
- Control for choosing between adder or memory input to the register(s)
- Control for sign or zero extending an immediate
- Control for the write enable to the memory

- The first control will either allow or block the output port (q) of the memory module to reach the input of the register file.
- The second control bit – controlling sign or zero extension – controls the immediate value. This immediate value is the base address offset when accessing an address in memory. So this control bit corresponds tot he addr port in the memory file.
- The third directly controls whether memory can be written to / overwritten. This directly maps to the “we” port of the memory file.

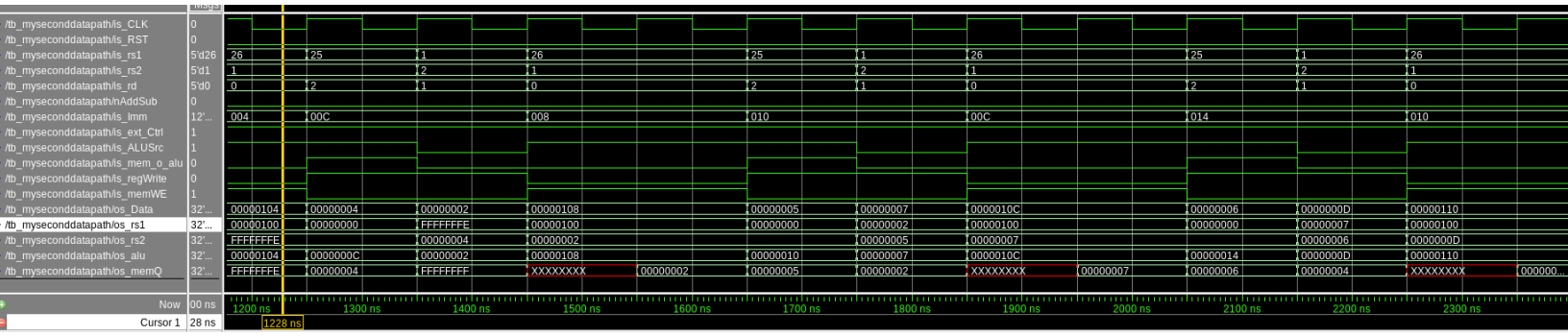
[Part 7 (b)] Draw a schematic of a simplified RISC-V processor consisting only of the base components used in part 2, the extender component described in part 4, and the data memory from part 3.



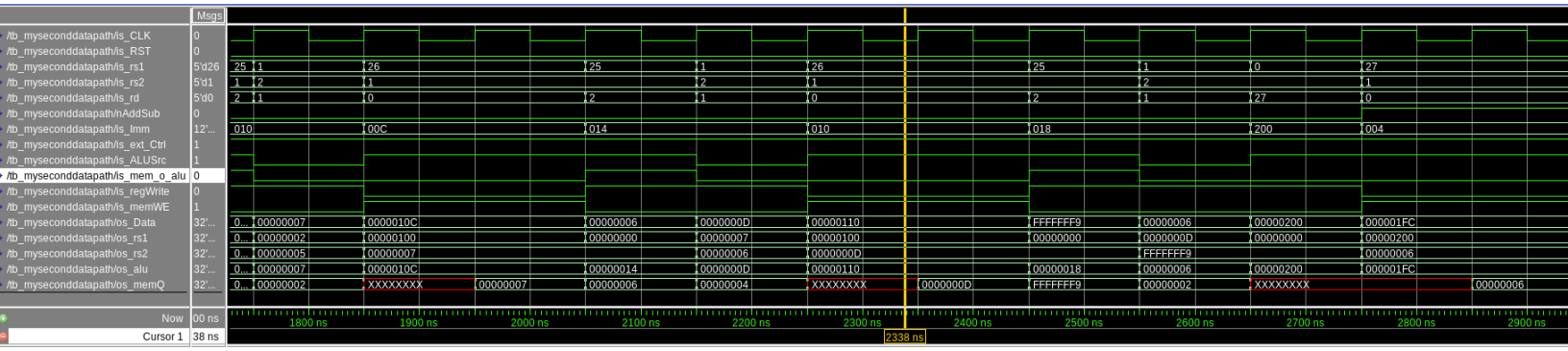
[Part 7 (c)] Waveform.



First screenshot ^^^^



Second screenshot ^^^^



Third screenshot ^^^^